

Rome Air Quality Index Prediction with AWS Lambda

Sannino Siria
(2001580)

Campos Gaiger Silveira Maria Beatriz
(2051307)

Zanoni Leila
(2033176)

June 8, 2026

1 Introduction

In the latest times, with the environmental conditions being a cause of concern for global wellness and biodiversity preservation, the Air Quality Index measurement has been taken more seriously into consideration as a pollution indicator. Recent AQI data indicates that air pollution is no longer limited to major industrial or metropolitan cities. Surprisingly, smaller towns and coastal regions are also appearing on the list of highly polluted locations. Our project deploys a prediction pipeline for the European AQI standard for Rome by using AWS Lambda, delivering real-time performance on CPU-only infrastructure through the use of cross-framework model compilation via the ONNX standard. We provide an extensive benchmark of both user-oriented and system-oriented metrics. We utilize the Locust testing framework to simulate realistic user loads in different configurations that allow us to show the vertical and horizontal scaling capabilities of our cloud-based solution.

2 Background

2.1 AQI evaluation

AQI is evaluated differently depending on the reference geographic area. For the scope of this project, we chose the European Air Quality Index (EAQI).

2.1.1 EAQI

The European Air Quality Index is comprised of six categories:

- Good - 1
- Fair - 2
- Moderate - 3
- Poor - 4
- Very Poor - 5
- Extremely Poor - 6

It is calculated starting from the concentration of the five most important air pollutants, namely, PM2.5, PM10, Nitrogen Dioxide, Ozone, and Sulfur Dioxide [1].

2.2 Model development pipeline

Given the task at hand, we decided to evaluate two different machine learning algorithms against two different, publicly available datasets to provide a comprehensive benchmark.

The two gradient boosting algorithms employed were **XGBoost** [2], which prioritizes stability on small datasets, and **LightGBM** [3], which prioritizes speed and memory efficiency. In all experiments, we chose to export the models to the ONNX [4] format to avoid the overhead of machine learning-related dependencies and for a lightweight deployment.

The datasets came from different sources; the baseline one is from Nitiraj Kulkarni and available on Kaggle [5], while the second one is taken from the official ARPA Lazio platform [6]. Both datasets provide pollutant measurements for Rome over a multiple-year span.

2.3 AWS Lambda

AWS Lambda is a serverless compute service that enables developers to execute code without having to manage servers. It provides various advantages, including automatic horizontal scaling and a cost-effective execution model that is triggered through events. Due to these characteristics, we decided that this was the best option as the compute engine of our infrastructure.

For this project, our machine learning inference code is deployed within Python 3.12 environments. Since the machine learning models we used are very lightweight, we were able to handle dependencies by having them locally compiled and later bundled into the function deployment packages using the AWS SAM CLI framework. This also allowed AWS to unpack and launch the container instantly, optimizing its initialization latencies.

2.4 CloudWatch

Amazon CloudWatch is automatically integrated with AWS Lambda, which forwards execution logs to Cloudwatch log groups without any additional configuration. We take advantage of this to obtain per-invocation metrics that we then use for our performance evaluation, specifically billed duration, peak

memory usage in relation to the provisioned allocation, and container initialization time (cold start overhead). These are discussed in detail in Section 5.

3 Application

Our implementation consists of a production-ready, multi-model serverless API designed to compute real-time air quality forecasts for Rome. The architecture is built entirely using the AWS Serverless Application Model (SAM), and exposes a public REST API through Amazon API Gateway that routes incoming traffic to four dedicated AWS Lambda functions.

Rather than concentrating all inference logic into a single function, we chose to isolate each model-dataset pairing into its own execution runtime. This modular approach allows downstream clients to target explicit predictive pairs via distinct POST endpoints, and enables independent scaling, deployment, and benchmarking of each model without interference.

3.1 Architecture

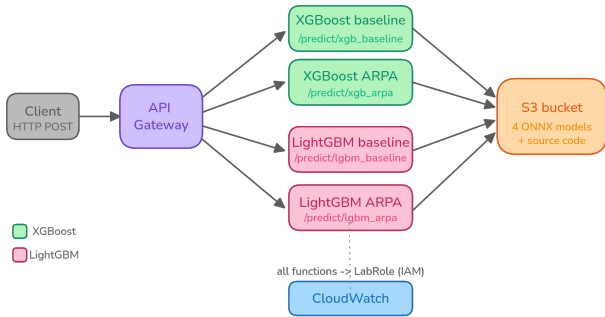


Figure 1: Architecture diagram

A diagram of the cloud architecture is shown in Fig 1. Incoming POST requests from a client are routed by API Gateway to one of four Lambda functions. Each function independently loads its ONNX model into memory at cold start and performs inference using the `onnxruntime` library. All models and source code are stored in an S3 bucket, a requirement imposed by AWS Lambda’s 50 MiB direct deployment limit. CloudWatch automatically captures execution logs and system metrics for every Lambda instance, which we leverage for our performance evaluation.

Typically, each Lambda function would have its own Identity and Access Management (IAM) role, with only the necessary permissions, abiding to the Principle of Least Privilege. However, The AWS Academy Learner Lab environment restricts IAM operations and does not allow the creation of new roles, because of this our deployment file was configured to avoid creating roles automatically. Instead, all Lambda functions use the existing `LabRole` provided by the AWS Academy environment, allowing us to

maintain an infrastructure-as-code deployment process while working within the constraints of the sandbox environment.

The Lambda memory allocation is parametrized in the SAM template, accepting values of 216 MiB, 512 MiB, or 1024 MiB, which we exploit during our vertical scaling experiments. The remaining configuration is fixed as shown in Table 1.

Table 1: Lambda Configuration

Parameter	Value
Runtime	Python 3.12
Timeout	15s
Default memory	512 MiB
Dependencies	<code>numpy</code> , <code>onnxruntime</code>

3.2 Endpoints and Input Schema

The API exposes four POST endpoints, each targeting a specific model-dataset pairing:

- `/predict/xgb_baseline`: Runs an XGBoost regressor trained on the baseline Kaggle dataset.
- `/predict/lgbm_baseline`: Runs a LightGBM regressor trained on the baseline Kaggle dataset.
- `/predict/xgb_arpa`: Runs an XGBoost classification pipeline trained on regional ARPA Lazio data.
- `/predict/lgbm_arpa`: Runs a LightGBM classification pipeline trained on regional ARPA Lazio data.

All endpoints accept a structured JSON payload under a `features` key. The baseline endpoints expect an 11-dimensional feature vector, while the ARPA Lazio endpoints require 12-dimensional vector mapping environmental pollutants and temporal identifiers in the following order:

$$\vec{x} = [\text{CO}_{\text{mean}}, \text{NO2}_{\text{max}}, \text{O3}_{\text{max}}, \text{O3}_{\text{mean}}, \\ \text{PM10}_{\text{mean}}, \text{PM2.5}_{\text{mean}}, \text{SO2}_{\text{max}}, \text{SO2}_{\text{mean}}, \\ \text{year}, \text{month}, \text{day}, \text{day_of_week}]$$

3.3 Execution Pipeline and Response Schema

Upon receiving a request, each Lambda handler parses the JSON body, reshapes the data using `numpy`, and passes it directly into an in-memory Open Neural Network Exchange (`onnxruntime`) inference session.

The response format varies based on the dataset. The baseline endpoints return a single continuous AQI prediction:

```
{
  "model": "XGBoost",
  "predicted_aqi": 34.125
}
```

The ARPA endpoints apply additional post-processing, rounding the raw regression output to a valid EAQI level (1–6) and mapping it to the corresponding EEA health category:

```
{
  "model": "LightGBM-EAQI-v1",
  "continuous_score": 2.7419,
  "eaqi_level": 3,
  "status": "Moderate"
}
```

All handlers include localized error handling that catches malformed input or type errors and returns an explicit 400 `Bad Request`, preventing unhandled Lambda crashes.

4 Performance Evaluation

4.1 Locust

Locust [\[7\]](#) is an open-source load testing framework designed for assessing the performance and scalability of web applications by simulating user behavior. It allows developers to define user scenarios and simulate concurrent users to measure how the application performs under load. Locust provides real-time monitoring of key performance metrics, such as response times, request rates, and error rates, making it a valuable tool for identifying bottlenecks and optimizing application performance. We use Locust to simulate realistic user behavior and measure the performance of our Lambda function under different load conditions.

4.2 Test Configurations

As shown in table [2](#), we worked on three different test scenarios to showcase the behavior of our application across different situations, like vertical scaling tests, by comparing the exact same loads across 256MiB, 512MiB, and 1024MiB memory configurations to measure compute latency and costs. Cold start penalty tests, by isolating the initial ramp-up period to measure the overhead of provisioning new Lambda containers, and finally a comparison test, running concurrent user scenarios explicitly against the single models to identify which ONNX runtime executes faster under pressure.

Table 2: Locust Load Testing Scenarios

Scenario	Target Models	RAM	Load
Vertical Scale	All Models	256, 512, 1024 MB	Baseline
Framework	All models separately	512 MB	50 Users
Cold Start	All models	512 MB	80 Users

5 Results

For each of these configurations, we report the results obtained by testing the different configurations mentioned in the previous section. By analyzing the aggregated statistics from Locust, we identified the main outcomes regarding vertical scaling efficiency, framework latency and cold start penalties.

For all of our scenarios, no failures were reported, confirming the robustness and correctness of our models.

5.1 Vertical Scaling and Cost Efficiency

To determine the optimal resource allocation for our task, we conducted identical baseline load tests across three memory configurations: 256MiB, 512MiB and 1024 MiB. Because AWS Lambda allocates CPU power linearly in proportion to RAM, this test measures the compute bottleneck. This test configuration shows us the average billed duration for each memory allocation:

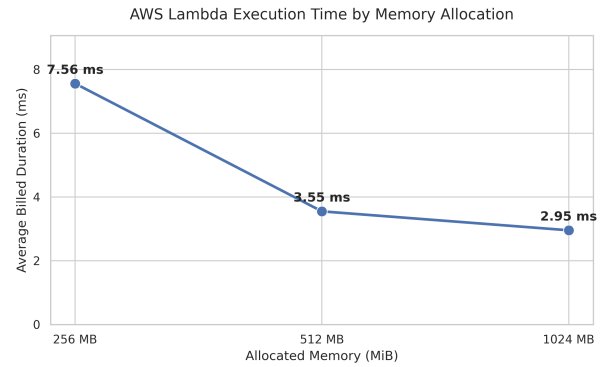


Figure 2: Average billed execution duration across different Lambda memory allocations

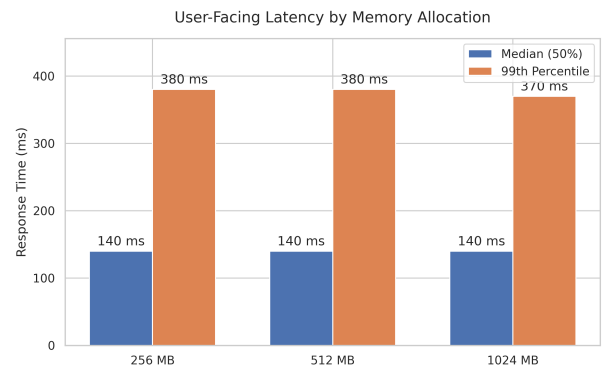


Figure 3: User Latency by Memory Allocation

These results indicate that the ONNX-optimized execution environment is highly efficient. The models compute predictions so rapidly that the bottleneck shifts entirely to network I/O and API Gateway routing overhead. As shown in Figure [3](#), the end-user median latency remained perfectly flat at exactly 140

ms, and the 99th percentile latency remained stable around 370-380 ms, regardless of whether 256 MiB or 1024 MiB of RAM was allocated.

5.2 Framework Performance Comparison

We isolated the XGBoost, LightGBM, XGBoost ARPA, LightGBM ARPA models and subjected each to a dedicated load test of 50 concurrent users to observe how the underlying framework behaves under sustained pressure.

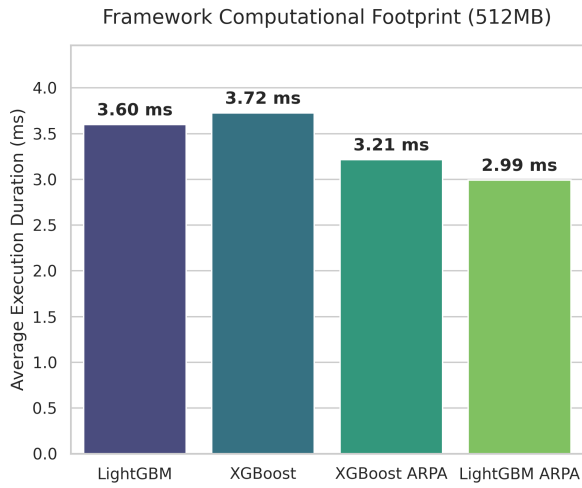


Figure 4: Comparison between frameworks

We can see in this figure that LightGBM ARPA is the fastest, with an average execution duration of 2.99 ms. We also observe that the two baseline models: XGBoost and LightGBM have similar durations, both trained on the same dataset.

5.3 Cold-Start penalty

While our median compute times were negligible, analyzing the maximum latencies highlights the inherent drawback of scale-to-zero serverless architectures: the "cold-start" penalty.

When a sudden burst of concurrent users hits the API, AWS must provision new Lambda containers, download the deployment package from S3, and load the ONNX model into memory.

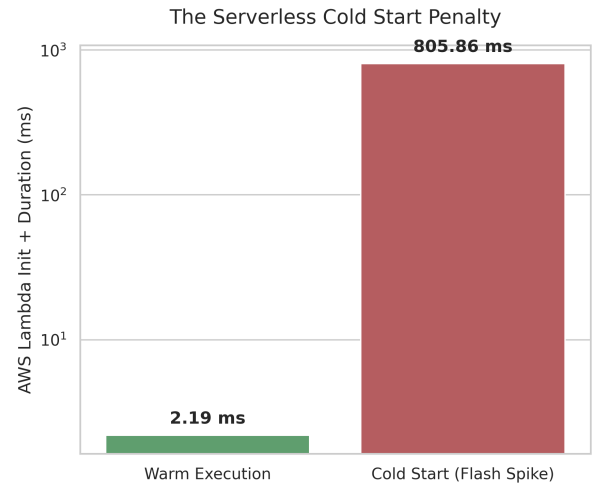


Figure 5: The initialization spike representing the container provisioning phase compared to warm executions.

Figure 5 visualizes the initialization phase duration. During severe cold-start spikes, CloudWatch captured maximum round-trip response times in the thousands of milliseconds as a heavy wave of users hit a completely idle system.

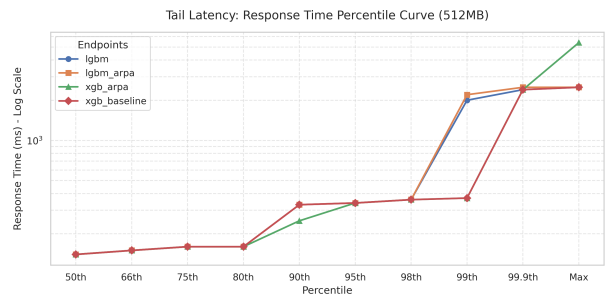


Figure 6: Tail Latency Curve proving that 99% of all requests are processed rapidly.

However, these extreme latencies are strictly isolated to the initial ramp-up phase. Figure 6 maps the comprehensive tail latency percentile curve on a logarithmic scale. This visual definitively proves that despite the initial anomalies caused by container cold starts, 99% of all requests are satisfied rapidly. Once the Lambda containers are warm, the serverless infrastructure provides highly reliable real-time predictions.

6 Conclusions

We have shown that gradient boosting models, when exported to the ONNX format and deployed on AWS Lambda, can serve real-time air quality predictions with negligible compute overhead. Across test scenarios, no failures were recorded, and inference times stayed in the low single-digit milliseconds regardless of memory allocation, meaning that the actual bottle-

neck is in network and API Gateway overhead rather than model computation.

Vertical scaling experiments revealed that increasing RAM from 256 MiB to 1024 MiB does not have a significant effect on user-facing latency, which remained flat at a 140 ms median across all configurations. This suggests that 256 MiB is sufficient for this workload, and over-provisioning memory offers no practical benefit. Among the four model-dataset pairings, LightGBM ARPA was the fastest with an average execution time of 2.99 ms, while the two base-

line models performed similarly to each other given that they share the same training data.

The one genuine drawback of this setup is the cold-start penalty. When the system is idle and a burst of users arrives, provisioning new containers pushed response times past 800 ms. This is an intrinsic property of scale-to-zero architectures and not something the application layer can avoid, though as the tail latency curve shows, it is strictly limited to the initial ramp-up phase.

7 Appendix

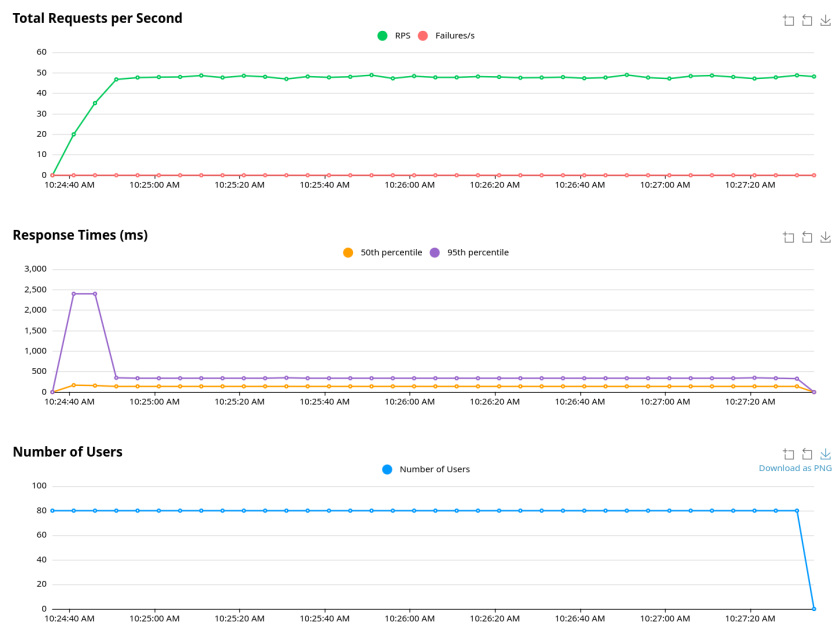


Figure 7: This is the cold penalty test, during which the initial diagonal ramp-up phase, concurrent users are incrementally spawned from 0 to a peak of 80 users. Once the maximum concurrency is reached, the sustained plateau phase demonstrates the warm execution environment, where the 80 active containers are continuously reused, stabilizing the response times.

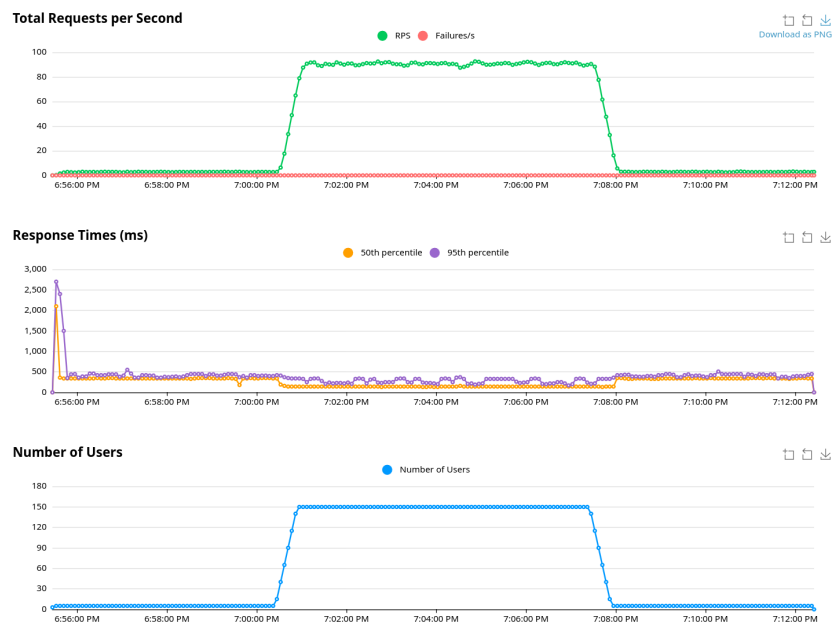


Figure 8: During the initial user ramp-up phase, the throughput increases progressively with the user concurrency. However, due to the proportional coupling of CPU performance with memory size in serverless environments, the 256 MiB tier exhibits a bounded processing ceiling, stabilizing at its maximum throughput plateau once full concurrency is reached.

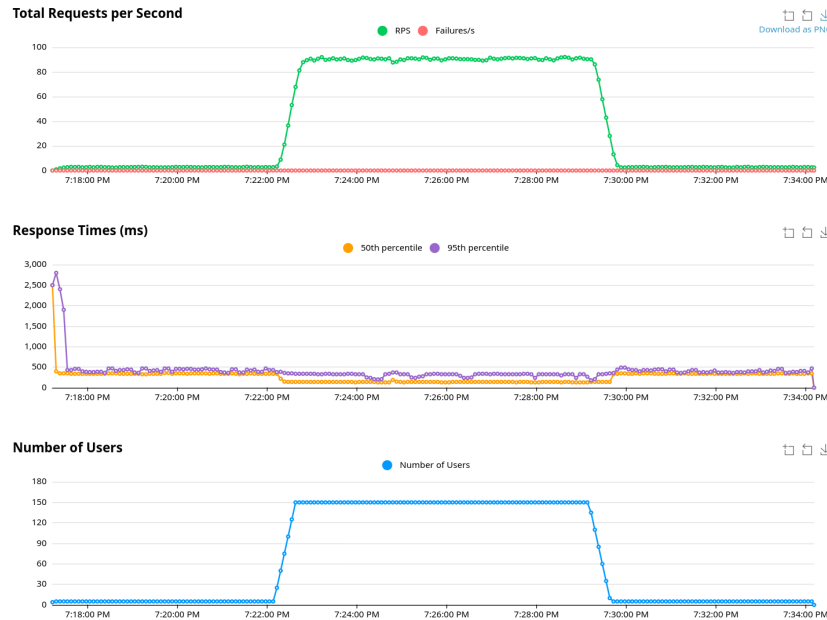


Figure 9: As observed in the system metrics, the Total Requests per Second (RPS) plateau remains nearly identical across the 256 MiB and 1024 MiB configurations. This demonstrates that even at the lowest memory tier, the Lambda architecture remains highly available and successfully processes all incoming requests without dropped connections under our maximum tested concurrency. However, while throughput remains constant, the execution speed scales dramatically.

References

- [1] “AQI Hub European Union (EAQI) — aqi-hub.info.” <https://aqihub.info/indices/eu> [Accessed 05-06-2026].
- [2] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” *CoRR*, vol. abs/1603.02754, 2016.
- [3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “Lightgbm: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems* (I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, eds.), vol. 30, Curran Associates, Inc., 2017.
- [4] ONNX Runtime developers, “Onnx runtime.” URL: <https://onnxruntime.ai/>, 2021. Version: 2.44.1.
- [5] N. Kulkarni, “Dataset: Air Quality Dataset for Rome — kaggle.com.” https://www.kaggle.com/datasets/nitirajkulkarni/rome-it-3169070?select=air_quality_historical.csv [Accessed 05-06-2026].
- [6] “Sistema di Monitoraggio - ARPA Lazio — arpalazio.it.” <https://www.arpalazio.it/web/guest/ambiente/aria/sistema-di-monitoraggio>. [Accessed 05-06-2026].
- [7] J. Heyman, L. Holmberg, *et al.*, “Locust - a modern load testing framework.” URL: <https://github.com/locustio/locust>, 2024. Version 2.28.1.